

Normalization

Normalizzazione 1NF

La prima forma normale è il principio di un valore per cella.

La prima regola dell'ordine è: una casella, un solo valore

La prima forma normale, detta 1NF, dice che ogni cella deve contenere un solo valore. Niente elenchi nascosti dentro una stessa colonna.

Pensa a una rubrica cartacea. Se nella casella "telefono" scrivi tre numeri separati da virgole, poi diventa difficile cercare, correggere o cancellare un solo numero.

Non bene:

id	nome	telefoni
1	Luca	123, 456

Qui la colonna `telefoni` contiene due valori insieme. Questo rende più difficile cercare, aggiornare e controllare i dati.

Come sistemarla

Hai due strade migliori:

- creare una riga per ogni telefono in una tabella separata
- oppure ripensare la struttura se il dato è stato modellato male

La 1NF è la base della pulizia. Se la salti, il resto del database parte già storto.

Meglio:

cliente_id	telefono
1	123
1	456

Ora ogni cella contiene un solo valore. Cercare il telefono **456** diventa molto più semplice.

:::note La 1NF non rende il database perfetto da sola. Però evita uno degli errori più comuni: infilare liste dentro una singola cella. :::

Normalizzazione 2NF

La seconda forma normale è l'idea di dipendere dall'intera chiave.

Qui entri in un concetto un po' più sottile

La seconda forma normale, o 2NF, dice che ogni colonna deve dipendere dall'intera chiave primaria, non solo da una parte.

Questo tema compare soprattutto quando la chiave primaria è composta da più colonne.

Pensa a una ricevuta con due informazioni che identificano una riga: **ordine_id** e **prodotto_id**. Il prezzo comprato in quella riga dipende da quella combinazione. Il nome del prodotto, invece, dipende solo dal prodotto.

Un modo semplice di pensarla

Se una tabella usa una chiave composta, ogni informazione non chiave dovrebbe avere senso solo guardando **tutta** la chiave.

Se basta un pezzo della chiave per determinare quel valore, probabilmente quel dato starebbe meglio in un'altra tabella.

Un esempio intuitivo

Immagina questa tabella:

ordine_id	prodotto_id	nome_prodotto	quantita
10	3	Tastiera	2

La quantità dipende dall'ordine e dal prodotto insieme. Il `nome_prodotto` , invece, dipende solo da `prodotto_id` .

In 2NF, il nome del prodotto dovrebbe stare nella tabella `prodotti` .

Perché serve

La 2NF aiuta a ridurre duplicazioni e anomalie negli aggiornamenti.

Non è il primo concetto di normalizzazione da memorizzare a forza. **Prima capisci bene 1NF e le relazioni. Poi questo livello diventa molto più naturale.**

Un consiglio pratico

Se la 2NF ti sembra astratta, non sei tu il problema. All'inizio succede spesso.

L'importante è capire l'idea generale: i dati devono vivere dove dipendono davvero.

Normalizzazione 3NF

La terza forma normale e l'idea di evitare dipendenze indirette.

Qui elimini i passaggi indiretti che sporcano il modello

La terza forma normale, o 3NF, dice che le colonne non chiave devono dipendere dalla chiave primaria, non da altre colonne non chiave.

In pratica: evita passaggi indiretti che creano duplicazioni e confusione.

Pensa a una scheda cliente con `cap` e `città` . Se la città dipende dal CAP, e il CAP dipende dal cliente, hai una dipendenza indiretta.

Un esempio intuitivo

Se in una tabella ordini salvi anche informazioni che dipendono dal cliente, e non dall'ordine, rischi di ripetere dati inutilmente.

Per esempio, se il nome della città dipende dal cliente, spesso non ha senso copiarlo dentro ogni ordine senza una buona ragione.

Un esempio in tabella

cliente_id	nome	cap	citta
1	Luca	00100	Roma
2	Marta	00100	Roma

Se `00100` indica sempre `Roma`, ripetere la città in ogni cliente può creare errori. Qualcuno potrebbe scrivere `Rma` in una riga e `Roma` in un'altra.

Una struttura più pulita può separare i CAP in una tabella dedicata.

Perché è utile

La 3NF riduce il rischio di incoerenze.

Se un dato vive nel posto giusto, quando cambia lo aggiorni una volta sola. **È questo uno dei vantaggi più grandi della normalizzazione.**

Un modo semplice per ricordarla

Se una colonna sembra dipendere più da un'altra colonna che dalla chiave della riga, fermati un attimo.

Potrebbe essere il segnale che quel dato sta nel posto sbagliato.

Denormalizzazione

Quando duplicare un po' di dati può migliorare letture o report.

A volte un po' di disordine controllato è una scelta intenzionale

La denormalizzazione è la scelta di duplicare alcuni dati per ottenere vantaggi pratici, spesso in velocità di lettura o semplicità di certe query.

È il contrario della normalizzazione pura, ma non è per forza sbagliata. Dipende dal problema che stai cercando di risolvere.

Pensa a una ricetta che copi su un foglio da tenere vicino ai fornelli. La ricetta originale resta nel libro, ma la copia ti fa risparmiare tempo mentre cucini.

Nel database, a volte copi un dato in più per leggere più in fretta.

Quando può avere senso

Può essere utile quando:

- hai report pesanti letti continuamente
- fai molte query complicate con join costose
- la velocità di lettura conta più della perfezione teorica

Qual è il prezzo da pagare

Il rischio è introdurre incoerenze.

Se un dato è copiato in più posti, quando cambia devi ricordarti di aggiornarlo ovunque. **La denormalizzazione va quindi fatta con un motivo preciso, non per comodità momentanea.**

Un esempio semplice

In una tabella `ordini` potresti salvare anche `nome_cliente`, oltre a `cliente_id`, per rendere un report più rapido da leggere.

```
SELECT numero_ordine, nome_cliente, totale
FROM ordini_report;
```

Questa scelta può essere utile per un report. Ma se il cliente cambia nome, devi sapere come aggiornare anche la copia.

:::tip Prima normalizza per capire bene i dati. Denormalizza solo quando hai un motivo concreto e misurabile. :::